

Presentation Guide — everything you need to defend this project

Read this top to bottom once. Part 1 assumes you know nothing about stock trading. Part 6 is the QA drill.

PART 1 — What a stock exchange actually is

The one-sentence version

A stock exchange is a **matchmaker for buyers and sellers of shares**. It does not set prices and it does not own anything. It keeps a list of who wants to buy and who wants to sell, and whenever a buyer's price meets a seller's price, it pairs them and records the trade. SeedPassw0rd!

The vocabulary you must be fluent in

Share (or stock) — a unit of ownership in a company. "10 shares of AAPL" means 10 units of Apple.

Symbol (or ticker) — the short code for a company. **AAPL** = Apple, SeedPassw0rd! **TSLA** = Tesla. Our system has 8 fixed symbols.

Order — an instruction: "I want to buy 10 AAPL" or "I want to sell 5 TSLA".

Side — **BUY** or **SELL**.

Bid — a buy order sitting in the market. **Ask** (or offer) — a sell order sitting in the market.

Limit order — "buy 10 AAPL, but pay **no more than** \$195.00". It has a price limit. If nobody will sell that cheap, the order waits.

Market order — "buy 10 AAPL at whatever the going rate is, right now". No price limit, so it fills immediately — but you don't control the price.

Resting order — an order that couldn't be matched immediately and is now sitting in the book, waiting.

The order book — the live list of all resting orders for one symbol, sorted by price. It looks like this:

```
      ASKS (people selling)
      196.09   90 shares
      195.89  170 shares
      195.70   95 shares    <- best ask (cheapest seller)
----- spread = 0.40
      195.30   95 shares    <- best bid (most generous buyer)
      195.11  180 shares
      194.91   90 shares
      BIDS (people buying)
```

Best bid — the highest price any buyer is offering. **Best ask** — the lowest price any seller is asking. **Spread** — the gap between them (here \$0.40).

Notice the book is never "crossed": the best bid is always *below* the best ask. If a buyer ever offered more than a seller wanted, they would have been matched instantly and neither would still be sitting there. **This invariant is a real correctness check in our tests.**

Fill — when your order gets matched. A **partial fill** is when only some of your quantity matched (you wanted 50, only 20 were available).

Trade (or execution) — the actual completed transaction between two orders.

Aggressor (or taker) — the order that arrives and crosses the spread to hit existing orders. **Maker** — the order that was already resting.

How matching works: price–time priority

When your buy order arrives, the engine looks at the sellers and picks:

1. **Best price first.** A seller asking 195.70 gets matched before one asking 195.89.
2. **Then earliest first.** Two sellers both asking 195.70? The one who placed their order first gets filled first. This is the "time" half, and it is why the book stores orders in a queue per price level.

That rule — **price first, then time** — is the single most important algorithm in the whole project.

Price improvement (the thing that confused our own test)

If you say "buy at up to 200.00" and the cheapest seller wants 195.70, **you pay 195.70, not 200.00**. The trade executes at the *resting* order's price. You named your worst acceptable price; you got a better one.

This matters in our system because the buyer had \$2,000 reserved (10×200) but only spent \$1,957 (10×195.70). The extra **\$43 must be given back**. If you remember one "gotcha" for QA, make it this one.

What "paper trading" means

Our exchange uses **virtual money**. Nobody deposits real cash and no real shares change hands. The mechanics are real; the money is fake. Think of it as a flight simulator for trading — same controls, no crashes.

The one thing a real exchange has that we don't

Real exchanges have an **issuance** step: a company IPOs and shares come into existence. We had to model this too, and it produced a genuinely interesting problem — see §5.6.

PART 2 — What our system does

The user journey (this is your demo script)

1. **Register / log in** → you get a token proving who you are.
2. **Deposit virtual funds** → your cash balance goes up.

3. **Browse the market** → see live prices and a candlestick price chart.
4. **Place an order** → buy or sell, limit or market.
5. **The engine matches it** → against other users' orders, not against a robot.
6. **Settlement happens** → your cash goes down, your shares go up, the seller's cash goes up, their shares go down.
7. **You see the result** → portfolio, profit/loss, and a notification.

The important claim, and the one the proposal is built on: **orders are matched against other real users**, not simulated. That is what makes this an exchange rather than a price-guessing game.

What a candlestick chart is

Each "candle" summarises one minute of trading with four numbers: **Open** (first trade price), **High**, **Low**, **Close** (last trade price). We build 1-minute and 5-minute candles. If a minute has no trades, we carry the previous close forward flat so the chart has no holes.

Profit and loss (P&L)

Average cost — what you paid per share on average. Buy 10 @ \$100 then 10 @ \$200 → you own 20 at an average cost of \$150.

Unrealized P&L — profit on paper. You still hold the shares. $(\text{current price} - \text{average cost}) \times \text{quantity}$.

Realized P&L — profit you actually locked in by selling. $(\text{sale price} - \text{average cost}) \times \text{quantity sold}$.

Selling does **not** change your average cost — it only converts unrealized profit into realized profit. That trips people up; know it.

PART 3 — The architecture

Why microservices at all (the question you WILL be asked)

The naive answer, "because the assignment said so", loses marks. The real answer:

We split by **what changes together and what scales together**. Each service was judged against one question: *does this responsibility have distinct data ownership, scaling needs, a security boundary, or a lifecycle?*

Concretely:

- **Market data is read 1000× more than it's written.** Every user watching a chart is reading constantly; writes only happen when a trade prints. That wants caching and a read replica — a completely different optimisation from the trading path.
- **Money is consistency-critical.** Cash handling needs locks and strict correctness. Making everything else pay that cost would be wasteful.
- **Matching is latency-critical and CPU-bound.** It wants to live in memory with a single writer and no database round trips.

- **Notifications must never slow down trading.** If email is slow, the exchange must not care.

Those are four genuinely different engineering problems. Putting them in one process means one deployment, one failure domain, and one set of trade-offs forced on all four.

The honest cost (say this — it shows maturity): microservices bought us independent scaling and isolation, and charged us **distributed-systems complexity**. We now need a saga instead of a database transaction, idempotent event handlers, and a circuit breaker. For a single-team project this is a real price. We paid it deliberately because it is what the course is about.

The seven services

Service	Port	Owns	Why separate
User	8001	identities, password hashes	Security boundary. Login traffic scales separately from trading.
Account	8002	cash, reservations, ledger	Most consistency-critical part. Isolates Redis locking.
Order	8003	order lifecycle, the saga	Orchestration concern, totally different from read paths.
Matching Engine	8004	order books (in memory)	Latency-sensitive, single-writer, no DB by design.
Market Data	8005	trades, candles, ticks	Extremely read-heavy → replica + caching.
Portfolio	8006	holdings, P&L, share reservations	A projection; only needs eventual consistency.
Notification	8007	alerts	Fire-and-forget; must never block trading.

Plus two **architectural components** that are not business services:

- **API Gateway (8000)** — one front door. Handles JWT validation, rate limiting and routing so seven services don't each reimplement them.
- **RabbitMQ** — the event backbone that decouples producers from consumers.

Database per service

Seven separate PostgreSQL containers, one per service. Not seven schemas in one database — seven actual database servers.

```

postgres-user           → user_db
postgres-account        → account_db
postgres-order          → order_db
postgres-market-primary → market_db   ┌ streaming replication
postgres-market-replica → market_db ─┘
postgres-portfolio      → portfolio_db
postgres-notification    → notification_db

```

The rule: no service ever reads another service's tables. The Order Service cannot `SELECT` from `account_db`. If it needs a balance it must *ask* the Account Service over HTTP, or learn it from an event.

Why this matters. Shared tables are the thing that quietly turns "microservices" back into a monolith: two services coupled through a schema can no longer be deployed, changed or scaled independently, and a migration by one team breaks the other. Separate databases make the boundary physically impossible to violate.

The cost, and the honest answer: you lose foreign keys and cross-service transactions. You cannot `JOIN` orders against users. That is exactly *why* we need the saga pattern (§5) and eventual consistency (§4.3).

The Matching Engine deliberately has **no database at all** — its order book is pure memory, because a disk write per order would destroy the latency that makes matching worth isolating.

PART 4 — How the services talk

Two mechanisms, chosen deliberately per situation.

4.1 Synchronous (REST over HTTP) — used when you need an answer *now*

Only two places:

- `Order` → `Account`: reserve funds before accepting a buy.
- `Order` → `Portfolio`: reserve shares before accepting a sell.

Why synchronous here: you cannot accept an order and *then* find out the user is broke. The answer gates the decision, so it must block.

Code: `libs/common/http_client.py`, called from `services/order-service/app/service.py:134` (`reserve`).

4.2 Asynchronous (events via RabbitMQ) — used for everything else

When something *has happened* and several parties need to know, we publish an event and move on.

Order Service	--order.accepted-->	Matching Engine
Matching Engine	--trade.executed-->	Order, Account, Portfolio, Market Data, Notification
Matching Engine	--order.cancelled-->	Order, Account, Portfolio, Notification

Why asynchronous here: one trade must update five services. Doing that with five blocking HTTP calls means the matching engine waits for the slowest one — so a slow notification service would slow down trading. With events, the engine publishes once and is immediately free.

This is the decoupling the proposal calls for: **a slow consumer can never slow down the producer.**

Full catalogue: [docs/events.md](#). Definitions: [libs/common/events.py](#).

4.3 Eventual consistency (know this term cold)

Because settlement happens via events, there is a window — a few hundred milliseconds — where the trade has executed but your portfolio hasn't updated yet.

That is **eventual consistency**: the system is guaranteed to become correct, not to be correct instantly. We accepted it for holdings and P&L, and we did *not* accept it for cash reservations (those are synchronous and locked).

Knowing *where* you allowed it and *why* is the mark of understanding it.

4.4 Service discovery

Services find each other by **name**, not IP: [http://account-service:8002](#). Docker Compose runs a DNS server on the [mse-net](#) network that resolves container names. Containers can be restarted, rescheduled or given new IPs and nothing breaks. That is the "Service Discovery" row in the proposal's tech stack.

PART 5 — The core workflow, step by step

This is the heart of your presentation. Walk through it slowly.

5.1 Placing a buy order — the happy path

User clicks Buy 10 AAPL @ 195.50.

Step 1 — Gateway ([services/gateway/app/main.py](#)) Validates the JWT, checks the rate limit (30 orders/min), forwards to Order Service.

Step 2 — Order Service validates ([services/order-service/app/service.py](#)) Symbol is real? Quantity a positive whole number? Price a multiple of \$0.01? Rejected here → 400, nothing persisted.

Step 3 — Persist as PENDING The order row is written *before* reserving money. We need the order ID to exist so the reservation can be tied to it and made idempotent.

Step 4 — Reserve the money (the «include» [Validate Buying Power](#) use case) Order calls Account: "hold \$1,955 for order X." Account takes a **Redis lock** on that user's balance, checks `cash - held ≥ amount`, and increases `held`. Not enough money → 409, order marked **REJECTED**.

Step 5 — Mark NEW and publish `order.accepted`.

Step 6 — Matching Engine matches ([services/matching-engine/app/engine.py:108](#)) It walks the sell side by price–time priority and pairs the order. For each pair it publishes a **`trade.executed`** event.

Step 7 — Five services react to that one event, in parallel:

- **Order** — marks the order **FILLED** or **PARTIALLY_FILLED**.
- **Account** — buyer's cash down, seller's cash up, hold discharged, any over-reservation released.
- **Portfolio** — buyer's shares up (recomputes average cost), seller's shares down (records realized P&L).

- **Market Data** — writes the trade, updates the 1m and 5m candles, caches the new price, broadcasts a tick to every connected browser.
- **Notification** — creates the "Order filled" alert for both sides.

Step 8 — The browser updates live over its WebSocket.

5.2 What happens when it fails — the saga

Steps 4–6 span three services. **You cannot wrap that in a database transaction**, because there are three separate databases. A **ROLLBACK** does not exist across a network.

So we use a **saga**: a sequence of local transactions, each with a **compensating action** that undoes it.

Step	Action	Compensation
T1	Reserve funds / shares	Release the reservation
T2	Persist order as NEW	Mark it REJECTED
T3	Publish <code>order.accepted</code>	pivot — no going back

The **pivot** is the point of no return. Once the matching engine has the order, it may match instantly, and you cannot un-trade. Everything before the pivot is reversible; nothing after it is. Real sagas are designed exactly this way.

The failure case to describe in QA: funds are reserved, then RabbitMQ is down so the publish fails. Without compensation the user's money stays locked forever for an order that will never exist. So we call release, mark the order REJECTED, and return 503. Code: `services/order-service/app/service.py:167` (`release`) and `:190`.

Compensation runs **blindly** — release is idempotent and returns "released 0" rather than 404 when nothing was held. That means the orchestrator never has to work out how far it got before failing. That is a deliberate design property.

5.3 Cancelling — and why it is two-phase

Naively: user cancels → release the money. **This is a race.** Between your decision to release and the release landing, the matching engine might fill that order. Now money has been given back for shares that were bought.

So cancellation goes **through the engine**:

1. Order Service marks it `CANCEL_PENDING` and publishes `order.cancel_requested`.
2. The **Matching Engine** — the only component that knows whether the order is still resting — removes it and publishes the authoritative `order.cancelled` with the exact unfilled quantity.
3. Then Account releases, and the Order Service marks it `CANCELLED`.
4. If it had already filled, the engine publishes `order.cancel_rejected` and the order goes back to `FILLED`.

This is why the UI shows "**Cancelling...**" rather than "Cancelled" — the request is accepted (HTTP 202), not completed. It can still come back filled.

This is a deliberate deviation from our own proposal, which had the Order Service publish `order.cancelled`. Say so, and say why: the book is the only authority on whether the order was still there. Owning a considered deviation scores better than pretending the first design was perfect.

5.4 Why a buy needs cash but a sell needs shares

Symmetry that is easy to miss. A buy reserves **money** in the Account Service. A sell reserves **shares** in the Portfolio Service.

Without share reservation, a user could sell 10 shares twice and end up owning -10 — a naked short. The `reserved_quantity` column and the DB constraint `reserved_quantity <= quantity` make that impossible.

Our proposal only specified funds locking. We added share reservation because the system is provably broken without it. That is a good thing to volunteer.

5.5 Self-trade prevention

If your own buy order would match your own sell order, the engine **skips** it. Otherwise you trade with yourself: your P&L becomes meaningless, and the Account Service would have to lock the same user twice inside one event — a deadlock. Code: `services/matching-engine/app/engine.py`, the `skipped` deque.

5.6 The bootstrap deadlock (a great story for QA)

When we first ran the system with real data, **not a single trade happened**.

Why: you can only sell shares you own, and you can only own shares by buying them. On a brand-new exchange nobody owns anything, so nobody can sell, so nobody can buy. Deadlock. Our logs showed hundreds of buy orders resting and every sell rejected with `insufficient shares` — which was the system working *correctly*.

Real exchanges solve this with **issuance** — a company IPOs and shares are created. We model that in `infra/seed/seed_demo_data.py`, which allocates initial holdings directly, then lets everything else flow through the public API.

If the examiner asks "did you actually test this properly?", this is the answer that proves you did.

PART 6 — Handling failure (the patterns question)

The examiner asked in the prompt: *what pattern did you use for a service failure and why?* There are four, each for a different failure mode. Know which is which.

6.1 Circuit Breaker — for a service that is DOWN

Problem we actually measured: with the order service not deployed, 40 concurrent requests to it made a *healthy* service unreachable for ~40 seconds.

Why: DNS lookups run in a shared thread pool. A lookup for a host that does not exist holds its thread far longer than our connect timeout. Enough doomed lookups and there are no threads left for the services that

are up. One dead service was taking the whole gateway with it — a **cascading failure**.

The pattern: after a failure, stop calling that service at all for 10 seconds. No socket, no DNS lookup, no thread consumed. It "fails fast" instead of failing slow.

Our breaker distinguishes two failure kinds, which is the detail worth showing:

- **Hard** (`ConnectError` — no such host): opens after **1** failure. On a Docker network this means the container isn't there. Retrying is pointless.
- **Soft** (`ConnectTimeout`): opens after **4**. A timeout might be temporary congestion — possibly caused by a *neighbouring* dead service — and locking out a healthy service over one blip is worse than the blip.

After the window, one probe is allowed through ("half-open"); success closes it.

Code: `services/gateway/app/proxy.py`. Measured result: collateral damage **40s** → **14s**, and zero once all services are deployed.

6.2 Bulkhead — to stop one failure spreading

Named after ship compartments: a hull breach floods one compartment, not the ship.

We cap in-flight requests **per upstream** at 8. One struggling service can consume at most 8 slots; the others always have capacity. Requests beyond the cap are shed immediately with 503 rather than queuing forever.

Code: same file, `MAX_INFLIGHT_PER_UPSTREAM`.

6.3 Retry with dead-letter queue — for a message that failed

If an event handler throws, the message must not be lost *and* must not block the queue forever.

Our approach: the message is republished to a `<queue>.retry` companion queue which holds it for 5 seconds (via RabbitMQ's message TTL) and then routes it back. **RabbitMQ does the waiting, so no consumer is blocked**. After 5 attempts it goes to a dead-letter queue for inspection.

The subtle part worth mentioning: the retry copy is published **before** the original is acknowledged. A crash in between causes a *duplicate*, which our idempotent handlers absorb. Acknowledging first would have caused a *lost event*. **Always prefer a duplicate over a loss** when money is involved.

Code: `libs/common/events.py`, `Broker.consume`.

6.4 Idempotency — because retries mean duplicates

At-least-once delivery means every handler will eventually see the same event twice. If a bill were applied twice, the user would be charged twice.

Every handler is **idempotent** — running it twice has the same effect as once. Implementation is **database-first**:

1. Check a Redis marker (fast path).
2. Do the work, and insert the `event_id` into a `processed_events` table **in the same transaction**. The primary key makes a duplicate impossible.
3. Write the Redis marker **only after** the commit.

Why that order matters (this is a genuinely good QA answer): the original code claimed the event in Redis *before* doing the work. If the process died in between, the redelivery saw "already processed", acknowledged, and the event was **silently lost forever**. Reversing the order means a crash costs one redundant retry instead of a lost trade.

6.5 Fail-closed vs fail-open (a judgement question)

Not every dependency failure should be treated the same:

- **Redis down + funds reservation → fail CLOSED.** Return 503, refuse the order. Reserving money without the lock could let a user spend the same cash twice.
- **Redis down + rate limiting → fail OPEN.** Allow the request. Rate limiting is protection, not correctness; taking the exchange down to enforce it is worse than the traffic.

Being able to explain *why the same failure gets opposite treatment* is the kind of thing that wins a QA exchange.

6.6 Health checks

Every service exposes two endpoints, and the difference is deliberate:

- `/health` — am I alive? **No dependency calls.** Docker polls it constantly.
- `/ready` — can I do useful work? Checks database, Redis and broker; returns 503 if degraded.

If `/health` checked the database, a brief DB blip would make Docker kill and restart a perfectly healthy container — making an outage worse.

PART 7 — The other required patterns

7.1 Distributed locking with Redis

Problem: a user opens two browser tabs and submits two orders at the same instant. Both check "do I have \$1,000?", both see yes, both proceed. The user spends \$2,000 they don't have. This is a **race condition**.

Solution: before touching a balance, acquire a lock in Redis: `lock:funds:{user_id}`. Only one request can hold it; the second waits.

Two implementation details worth knowing:

1. **The lock has a 5-second TTL.** If the holder crashes, the lock expires instead of jamming the system forever.
2. **Release uses a Lua compare-and-delete.** Each holder stores a random token and deletes the key *only if the token still matches*. Otherwise a slow request whose lock already expired could delete the lock a *different* request now legitimately holds.

Code: `libs/common/redis_client.py`, wrapped in `services/account-service/app/locking.py`.

Belt and braces: we also use `SELECT ... FOR UPDATE` (a database row lock) and `CHECK` constraints. The Redis lock is for throughput; the database is the ultimate guarantee. If a bug ever tried to make a balance

negative, Postgres rejects the transaction. Code: `services/account-service/app/repo.py:19`.

7.2 CQRS — Command Query Responsibility Segregation

The idea: **separate the model you write with from the model you read from**.

- **Write side:** orders and trades. Normalised, transactional, correctness-first.
- **Read side:** the Portfolio Service. It answers "what do I own and what is it worth?" instantly, without joining across orders and trades.

Portfolio holds no authoritative data — it is a **projection**, rebuilt from `trade.executed` events. The price: it is eventually consistent (§4.3). The benefit: the read is a single indexed lookup instead of an aggregation over every trade you ever made.

Code: `services/portfolio-service/app/events.py:79` (average cost) and `:90` (realized P&L).

7.3 Master–slave replication (the graded showpiece)

The Market Data database runs **two Postgres instances**:

- **Primary** — accepts writes (trades, candle updates).
- **Replica** — a continuously updated copy that accepts **reads only**.

Postgres streams its write-ahead log (WAL) from primary to replica, so the replica is a live copy, typically milliseconds behind.

Why here specifically: market data is the read-heavy path — every chart, every history query. Sending those to the replica keeps the primary free for trade ingestion. Browsing traffic then scales independently of trading traffic.

How to prove it live in the demo:

```
docker compose exec postgres-market-primary psql -U mse -d market_db -c "SELECT
client_addr, state FROM pg_stat_replication;"
```

→ one row, `streaming`.

```
docker compose exec postgres-market-replica psql -U mse -d market_db -c "SELECT
pg_is_in_recovery();"

```

→ `t` (true = it is a standby, not a normal database).

Writing to the replica fails with `cannot execute INSERT in a read-only transaction` — which is the proof it is a genuine standby and not just a second copy.

The trade-off (volunteer this): replica lag means a chart read immediately after a trade can miss the newest candle. We mitigate it in the UI by also applying the live WebSocket tick, so the user never sees stale data. If the replica is unreachable the service falls back to the primary rather than failing.

Code: `infra/postgres/`, and the two engines at the top of `services/market-data-service/app/main.py`.

7.4 API Gateway responsibilities

- **Routing** — strips `/api`, longest prefix wins.
- **Authentication** — validates the JWT and forwards it unchanged, so each service validates it again (defence in depth).
- **Rate limiting** — Redis fixed-window; 30/min on order placement, 120/min otherwise.
- **Security** — any path containing `/internal/` returns **404**. Internal endpoints are reachable only from inside the Docker network. Client-supplied `X-Internal-Key` and `X-User-Id` headers are **stripped**, so a client cannot forge either.
- **WebSocket proxying** — bridges the live tick feed so the gateway stays the single entry point.

7.5 JWT authentication

A JWT is a **signed** token containing your user ID, email and an expiry. Signed, not encrypted — anyone can read it, nobody can forge it without the secret.

Every service holds the same `JWT_SECRET`, so any service can verify a token without calling the User Service. That removes a network hop from every single request.

The operational trap: if one container has a different secret, every request 401s with no useful message. So each service logs a **fingerprint** (a hash prefix) of its secret at startup, and `verify_stack.py` asserts they all match.

7.6 Money handling (a favourite examiner question)

Never use floating point for money. $0.1 + 0.2 = 0.30000000000000004$. Over thousands of trades those errors accumulate into real discrepancies.

Our rules, enforced in `libs/common/money.py`:

- `Decimal` everywhere, 4 decimal places, `ROUND_HALF_UP`.
- Stored as `NUMERIC(18,4)` — an exact decimal type, not a float.
- Transported as a **JSON string** ("`195.5000`"), because JSON numbers are doubles and would reintroduce the error at the boundary.
- `to_money()` **actively rejects** float input rather than silently converting.

One documented exception: `/market/candles` returns numbers, because the charting library requires them. Deliberate, and written down.

PART 8 — Where everything lives in code

Concept	File
Event definitions, broker, retry/DLQ	<code>libs/common/events.py</code>

Concept	File
Distributed lock, idempotency helpers	<code>libs/common/redis_client.py</code>
Money rules	<code>libs/common/money.py</code>
JWT	<code>libs/common/security.py</code>
Matching algorithm	<code>services/matching-engine/app/engine.py:108</code>
Cancel from the book	<code>services/matching-engine/app/engine.py:178</code>
The saga	<code>services/order-service/app/service.py</code>
Reserve / compensating release	<code>service.py:134</code> / <code>service.py:167</code>
Funds reservation + locking	<code>services/account-service/app/routes.py:130</code> , <code>app/locking.py</code>
Row lock (FOR UPDATE)	<code>services/account-service/app/repo.py:19</code>
Average cost / realized P&L	<code>services/portfolio-service/app/events.py:79</code> , <code>:90</code>
Candle aggregation	<code>services/market-data-service/app/events.py</code>
Read/write split	top of <code>services/market-data-service/app/main.py</code>
Circuit breaker + bulkhead	<code>services/gateway/app/proxy.py</code>
Rate limiting	<code>services/gateway/app/ratelimit.py</code>
Replication setup	<code>infra/postgres/</code>

PART 9 — QA drill

"Why microservices? Isn't this over-engineered for a course project?" Yes, for a project this size a monolith would ship faster — and we'd say so. We chose it because four parts of this system have genuinely different needs: market data is read-heavy, matching is latency-critical, cash is consistency-critical, notifications are fire-and-forget. We paid the complexity cost deliberately and can point at exactly what it bought and what it charged us.

"How do you handle a transaction across services without a database transaction?" A saga. Three local transactions, each with a compensating action, and a pivot point after which no rollback is possible. Walk them through §5.2.

"What if a service goes down mid-trade?" Depends where. Before the pivot: compensation releases the reservation and the order is rejected. After the pivot: the trade is durable in RabbitMQ and the consumer processes it when it comes back — that's what durable queues are for. And the circuit breaker stops the dead service degrading the healthy ones.

"How do you know an event isn't processed twice?" Idempotency: `processed_events` primary key written in the same transaction as the work, Redis as a fast path *after* the commit. Explain why that order matters

(\$6.4).

"Show me the replication actually working." Run the two commands in §7.3, then write on the primary and read it back from the replica, then try to write to the replica and show it refuse.

"Why is the portfolio sometimes a second behind?" Eventual consistency, by design — it's a CQRS projection built from events. Cash reservations are *not* eventually consistent; those are synchronous and locked. Naming where you allowed it and where you didn't is the answer.

"What happens if two people buy the same last share at the same time?" Price–time priority: whoever's order reached the engine first gets it. The engine is single-writer per symbol, so there is no ambiguity. The second order rests or is cancelled.

"Can a user spend the same money twice?" No — three layers. Redis distributed lock serialises the requests, a `SELECT FOR UPDATE` row lock serialises the database access, and a `CHECK` constraint makes a negative balance impossible even if the code is wrong.

"Can a user sell shares they don't own?" No. Sells reserve shares in the Portfolio Service first, and the DB enforces `reserved_quantity <= quantity`. We added this because the original proposal only specified funds locking.

"Why does the matching engine have no database?" Latency. Matching is the hot path; a disk write per order would defeat the point of isolating it. The trade-off is that a restart loses resting orders, so it rebuilds the book from the Order Service's open orders on boot.

"What's the weakest part of your system?" Have an answer ready — refusing to name one looks worse than naming one. Honest options: the in-memory order book is a single point of failure with no horizontal scaling; there is no reaper for reservations if the Order Service dies permanently between reserving and publishing; `processed_events` grows unbounded.

"Did you test it, or does it just run?" 13 test suites, 0 failures: a contract suite over the shared money/event rules, all 23 gateway routes, a live suite that publishes a real event through RabbitMQ and reads the result back over HTTP, an end-to-end business-flow smoke test, and eight per-service self-checks. Plus the bootstrap-deadlock story in §5.6 as proof we tested with real volume, not a toy case.

PART 10 — Demo running order

1. `docker compose ps` — 18 containers healthy.
2. Show the **replication proof** (§7.3). Do this early; it's a graded item.
3. Show **RabbitMQ management UI** — 15 queues, bindings, message rates.
4. Show **Swagger** on any service — auto-generated API docs.
5. In the app: log in as `demo@mse.local` / `DemoPassw0rd!`.
6. Deposit funds → show the wallet ledger (`HOLD`, `TRADE_BUY`, `RELEASE`).
7. Open a market page — chart, order book with spread, live trade tape.
8. **Place a buy order** and narrate the whole \$5.1 flow as it happens.
9. Show the fill, the portfolio update, the notification.
10. Place a limit order far from the market, then **cancel it** — point out the "Cancelling..." state and explain two-phase cancel.

11. Try to **sell shares you don't own** → 409, explain share reservation.
12. Finish with `python scripts/smoke_test.py` — 25 checks, all green.

Keep the failure stories in your pocket: the cascading failure and circuit breaker (§6.1), the bootstrap deadlock (§5.6), and the idempotency ordering bug (§6.4). Examiners reward candidates who found real problems and can explain the fix.